

scipy.spatial.distance.cdist

`scipy.spatial.distance.cdist(XA, XB, metric='euclidean', *, out=None, **kwargs)` [\[source\]](#)

Compute distance between each pair of the two collections of inputs.

See Notes for common calling conventions.

Parameters: **XA : array_like**

An m_A by n array of m_A original observations in an n -dimensional space. Inputs are converted to float type.

XB : array_like

An m_B by n array of m_B original observations in an n -dimensional space. Inputs are converted to float type.

metric : str or callable, optional

The distance metric to use. If a string, the distance function can be 'braycurtis', 'canberra', 'chebyshev', 'cityblock', 'correlation', 'cosine', 'dice', 'euclidean', 'hamming', 'jaccard', 'jensenshannon', 'kulczynski1', 'mahalanobis', 'matching', 'minkowski', 'rogerstanimoto', 'russellrao', 'seuclidean', 'sokalmichener', 'sokalsneath', 'sqeuclidean', 'yule'.

****kwargs : dict, optional**

Extra arguments to *metric*: refer to each metric documentation for a list of all possible arguments.

Some possible arguments:

p : scalar The p-norm to apply for Minkowski, weighted and unweighted. Default: 2.

w : array_like The weight vector for metrics that support weights (e.g., Minkowski).

V : array_like The variance vector for standardized Euclidean. Default: var(vstack([XA, XB]), axis=0, ddof=1)

VI : array_like The inverse of the covariance matrix for Mahalanobis. Default:

inv(cov(vstack([XA, XB].T))).T

out : ndarray The output array If not None, the distance matrix Y is stored in this array.

Returns: **Y : ndarray**

A m_A by m_B distance matrix is returned. For each i and j , the metric `dist(u=XA[i], v=XB[j])` is computed and stored in the ij th entry.

Raises: **ValueError**

An exception is thrown if XA and XB do not have the same number of columns.

Notes

The following are common calling conventions:

1. `Y = cdist(XA, XB, 'euclidean')`

Computes the distance between m points using Euclidean distance (2-norm) as the distance metric between the points. The points are arranged as m n -dimensional row vectors in the matrix X.

2. `Y = cdist(XA, XB, 'minkowski', p=2.)`

Computes the distances using the Minkowski distance $\|u - v\|_p$ (p -norm) where $p > 0$ (note that this is only a quasi-metric if $0 < p < 1$).

3. `Y = cdist(XA, XB, 'cityblock')`

Computes the city block or Manhattan distance between the points.

4. `Y = cdist(XA, XB, 'seuclidean', V=None)`

Computes the standardized Euclidean distance. The standardized Euclidean distance between two n-vectors $\mathbf{u}$ and $\mathbf{v}$ is

$$\sqrt{\sum (u_i - v_i)^2 / V[x_i]}.$$

V is the variance vector; $V[i]$ is the variance computed over all the i 'th components of the points. If not passed, it is automatically computed.

5. `Y = cdist(XA, XB, 'sqeuclidean')`

Computes the squared Euclidean distance $\|\mathbf{u} - \mathbf{v}\|_2^2$ between the vectors.

6. `Y = cdist(XA, XB, 'cosine')`

Computes the cosine distance between vectors $\mathbf{u}$ and $\mathbf{v}$,

$$1 - \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$$

where $\|\mathbf{*}\|_2$ is the 2-norm of its argument $\mathbf{*}$, and $\mathbf{u} \cdot \mathbf{v}$ is the dot product of $\mathbf{u}$ and $\mathbf{v}$.

7. `Y = cdist(XA, XB, 'correlation')`

Computes the correlation distance between vectors $\mathbf{u}$ and $\mathbf{v}$. This is

$$1 - \frac{(\mathbf{u} - \bar{\mathbf{u}}) \cdot (\mathbf{v} - \bar{\mathbf{v}})}{\|(\mathbf{u} - \bar{\mathbf{u}})\|_2 \|(\mathbf{v} - \bar{\mathbf{v}})\|_2}$$

where $\bar{\mathbf{v}}$ is the mean of the elements of vector $\mathbf{v}$, and $\mathbf{x} \cdot \mathbf{y}$ is the dot product of $\mathbf{x}$ and $\mathbf{y}$.

8. `Y = cdist(XA, XB, 'hamming')`

Computes the normalized Hamming distance, or the proportion of those vector elements between two n-vectors $\mathbf{u}$ and $\mathbf{v}$ which disagree. To save memory, the matrix $\mathbf{X}$ can be of type boolean.

9. `Y = cdist(XA, XB, 'jaccard')`

Computes the Jaccard distance between the points. Given two vectors, $\mathbf{u}$ and $\mathbf{v}$, the Jaccard distance is the proportion of those elements $\mathbf{u}[i]$ and $\mathbf{v}[i]$ that disagree where at least one of them is non-zero.

10. `Y = cdist(XA, XB, 'jensenshannon')`

Computes the Jensen-Shannon distance between two probability arrays. Given two probability vectors, $\mathbf{p}$ and $\mathbf{q}$, the Jensen-Shannon distance is

$$\sqrt{\frac{D(\mathbf{p} \parallel \mathbf{m}) + D(\mathbf{q} \parallel \mathbf{m})}{2}}$$

where $\mathbf{m}$ is the pointwise mean of $\mathbf{p}$ and $\mathbf{q}$ and D is the Kullback-Leibler divergence.

11. `Y = cdist(XA, XB, 'chebyshev')`

Computes the Chebyshev distance between the points. The Chebyshev distance between two n-vectors $\mathbf{u}$ and $\mathbf{v}$ is the maximum norm-1 distance between their respective elements. More precisely, the distance is given by

$$d(\mathbf{u}, \mathbf{v}) = \max_i |u_i - v_i|.$$

12. `Y = cdist(XA, XB, 'canberra')`

Computes the Canberra distance between the points. The Canberra distance between two points $\mathbf{u}$ and $\mathbf{v}$ is

$$d(\mathbf{u}, \mathbf{v}) = \sum_i \frac{|u_i - v_i|}{|u_i| + |v_i|}.$$

13. `Y = cdist(XA, XB, 'braycurtis')`

Computes the Bray-Curtis distance between the points. The Bray-Curtis distance between two points `u` and `v` is

$$d(u, v) = \frac{\sum_i (|u_i - v_i|)}{\sum_i (|u_i + v_i|)}$$

14. `Y = cdist(XA, XB, 'mahalanobis', VI=None)`

Computes the Mahalanobis distance between the points. The Mahalanobis distance between two points `u` and `v` is $\sqrt{(u - v)(1/V)(u - v)^T}$ where $(1/V)$ (the `VI` variable) is the inverse covariance. If `VI` is not None, `VI` will be used as the inverse covariance matrix.

15. `Y = cdist(XA, XB, 'yule')`

Computes the Yule distance between the boolean vectors. (see [yule](#) function documentation)

16. `Y = cdist(XA, XB, 'matching')`

Synonym for 'hamming'.

17. `Y = cdist(XA, XB, 'dice')`

Computes the Dice distance between the boolean vectors. (see [dice](#) function documentation)

18. `Y = cdist(XA, XB, 'kulczynski1')`

Computes the kulczynski distance between the boolean vectors. (see [kulczynski1](#) function documentation)

19. `Y = cdist(XA, XB, 'rogerstanimoto')`

Computes the Rogers-Tanimoto distance between the boolean vectors. (see [rogerstanimoto](#) function documentation)

20. `Y = cdist(XA, XB, 'russellrao')`

Computes the Russell-Rao distance between the boolean vectors. (see [russellrao](#) function documentation)

21. `Y = cdist(XA, XB, 'sokalmichener')`

Computes the Sokal-Michener distance between the boolean vectors. (see [sokalmichener](#) function documentation)

22. `Y = cdist(XA, XB, 'sokalsneath')`

Computes the Sokal-Sneath distance between the vectors. (see [sokalsneath](#) function documentation)

23. `Y = cdist(XA, XB, f)`

Computes the distance between all pairs of vectors in X using the user supplied 2-arity function f. For example, Euclidean distance between the vectors could be computed as follows:

```
dm = cdist(XA, XB, lambda u, v: np.sqrt(((u-v)**2).sum()))
```

Note that you should avoid passing a reference to one of the distance functions defined in this library. For example,:

```
dm = cdist(XA, XB, sokalsneath)
```

would calculate the pair-wise distances between the vectors in X using the Python function [sokalsneath](#). This would result in sokalsneath being called $\binom{n}{2}$ times, which is inefficient. Instead, the optimized C version is more efficient, and we call it using the following syntax:

```
dm = cdist(XA, XB, 'sokalsneath')
```

Q Search the docs ...

- [Clustering package](#)
([_scipy.cluster_](#))
- [K-means clustering and vector quantization](#)
([_scipy.cluster.vq_](#))
- [Hierarchical clustering](#)
([_scipy.cluster.hierarchy_](#))
- [Constants](#)
([_scipy.constants_](#))
- [Datasets](#) ([_scipy.datasets_](#))
- [Discrete Fourier transforms](#)
([_scipy.fft_](#))
- [Legacy discrete Fourier transforms](#) ([_scipy.fftpack_](#))
- [Integration and ODEs](#)
([_scipy.integrate_](#))
- [Interpolation](#)
([_scipy.interpolate_](#))
- [Input and output](#) ([_scipy.io_](#))
- [Linear algebra](#)
([_scipy.linalg_](#))
- [Low-level BLAS functions](#)
([_scipy.linalg.blas_](#))

Examples

[Low-level LAPACK functions](#)
([_scipy.linalg.lapack_](#))

[BLAS Functions for Cython](#)

[LAPACK functions for Cython](#)

[Interpolative matrix decomposition](#)
([_scipy.linalg.interpolative_](#))

[Miscellaneous routines](#)
([_scipy.misc_](#))

[Multidimensional image processing](#) ([_scipy.ndimage_](#))

[Orthogonal distance regression](#)
([_scipy.odr_](#))

[Optimization and root finding](#)
([_scipy.optimize_](#))

[Cython optimize zeros API](#)

[Signal processing](#)
([_scipy.signal_](#))

[Sparse matrices](#)
([_scipy.sparse_](#))

[Sparse linear algebra](#)
([_scipy.sparse.linalg_](#))

[Compressed sparse graph routines](#)
([_scipy.sparse.csgraph_](#))

[Spatial algorithms and data structures](#) ([_scipy.spatial_](#))

Find the Euclidean distances between four 2-D coordinates:

```
>>> from scipy.spatial import distance
>>> coords = [(35.0456, -85.2672),
...          (35.1174, -89.9711),
...          (35.9728, -83.9422),
...          (36.1667, -86.7833)]
>>> distance.cdist(coords, coords, 'euclidean')
array([[ 0.      ,  4.7044,  1.6172,  1.8856],
       [ 4.7044,  0.      ,  6.0893,  3.3561],
       [ 1.6172,  6.0893,  0.      ,  2.8477],
       [ 1.8856,  3.3561,  2.8477,  0.      ]])
```

Find the Manhattan distance from a 3-D point to the corners of the unit cube:

```
>>> a = np.array([[0, 0, 0],
...              [0, 0, 1],
...              [0, 1, 0],
...              [0, 1, 1],
...              [1, 0, 0],
...              [1, 0, 1],
...              [1, 1, 0],
...              [1, 1, 1]])
>>> b = np.array([[ 0.1,  0.2,  0.4]])
>>> distance.cdist(a, b, 'cityblock')
array([[ 0.7],
       [ 0.9],
       [ 1.3],
       [ 1.5],
       [ 1.5],
       [ 1.7],
       [ 2.1],
       [ 2.3]])
```

[Previous](#)
[scipy.spatial.distance.pdist](#)

[Next](#)
[scipy.spatial.distance.squareform](#)